

Assistant hỏi đáp thủ tục hành chính công Việt Nam ứng dụng Agentic RAG

Nguyễn Xuân Phúc¹, Nguyễn Minh Triết¹, Đỗ Trọng Hợp^{1,*}, Nguyễn Ngọc Quý^{1,*}

¹Trường Đại học Công nghệ Thông tin, DHQG-HCM, TP. Hồ Chí Minh

Abstract

Thủ tục hành chính công ở Việt Nam trải rộng trên hàng nghìn quy trình, mỗi quy trình có thành phần hồ sơ, trình tự, lệ phí, thời hạn và căn cứ pháp lý riêng, lại thường xuyên thay đổi theo văn bản mới — khiến người dân và doanh nghiệp rất khó tra cứu chính xác. Báo cáo này trình bày một trợ lý hỏi–đáp tiếng Việt cho miền thủ tục hành chính công, dựa trên kiến trúc *Agentic* RAG (Retrieval-Augmented Generation). Kho tri thức gồm **5.208 thủ tục** được thu thập tự động từ Cổng Dịch vụ công Quốc gia (dichvucong.gov.vn), chuẩn hoá thành corpus Markdown có metadata và nhúng bằng Azure `text-embedding-3-large` (3072 chiều) vào kho vector lai ghép (Chroma + LanceDB). Lỗi suy luận là một tác tử **ReAct** hai pha: pha một, agent chủ động tra cứu corpus trước rồi mới bổ sung tìm kiếm web khi cần (tinh thần *Corrective*-RAG); pha hai, hệ thống tổng hợp câu trả lời có **trích dẫn nội dung** và **tô sáng** đúng đoạn bằng chứng. Hệ thống còn trực quan hoá quá trình suy luận của agent theo thời gian thực, gợi ý câu hỏi tiếp theo, và sinh sơ đồ tư duy cho từng thủ tục, được tích hợp qua backend FastAPI (streaming SSE) và giao diện React. Báo cáo mô tả động lực, các công trình liên quan, và kế hoạch triển khai bốn tuần; phần phương pháp chi tiết và đánh giá sẽ được hoàn thiện ở các bước tiếp theo, cùng với việc trừu tượng hoá tầng LLM để chạy được trên nhiều nhà cung cấp (Azure, OpenAI, và mô hình mã nguồn mở phục vụ qua vLLM).

1 Introduction

1.1 Bối cảnh và động lực

Cải cách thủ tục hành chính là một trong những trụ cột của tiến trình xây dựng Chính phủ số ở Việt Nam. Tuy nhiên, hệ thống thủ tục hành chính công hiện hành gồm hàng nghìn thủ tục thuộc nhiều lĩnh vực và nhiều cấp thực hiện (xã/phường, tỉnh/thành phố, bộ/trung ương); mỗi thủ tục lại bao gồm thành phần hồ sơ, trình tự thực hiện, cách thức nộp, lệ phí, thời hạn giải quyết và căn cứ pháp lý riêng. Các thông tin này phân tán, được diễn đạt bằng văn phong pháp lý, và thường xuyên thay đổi theo các thông tư, nghị định mới. Hệ quả là người dân và doanh nghiệp dễ nộp sai cơ quan, thiếu giấy tờ, phải đi lại nhiều lần — gây lãng phí thời gian và chi phí xã hội.

Cổng Dịch vụ công Quốc gia [14] tập trung dữ liệu thủ tục, nhưng việc tra cứu chủ yếu dựa trên *từ khoá*: người dùng phải biết gần đúng tên thủ tục, và hệ thống không hỗ trợ hỏi–đáp bằng ngôn ngữ tự nhiên, không tổng hợp hay đối chiếu thông tin giữa các thủ tục liên quan. Các mô hình ngôn ngữ lớn (LLM) như GPT-4 [10] giải quyết tốt phần giao tiếp ngôn ngữ tự nhiên, nhưng khi hỏi về dữ kiện hành chính cụ thể (mã mẫu đơn, số văn bản, mức lệ phí, thành phần hồ sơ) chúng dễ *ảo giác* — sinh ra thông tin nghe hợp lý nhưng sai [5]. Trong một miền đòi hỏi độ chính xác cao và phải truy được nguồn như thủ tục hành chính, rủi ro này là không chấp nhận được.

* Giảng viên hướng dẫn

Retrieval-Augmented Generation (RAG) nối LLM với một kho tri thức ngoài, giúp giảm ảo giác và cập nhật thông tin mà không cần huấn luyện lại [7, 3]. Tuy vậy, RAG *tĩnh* — truy hỏi một lần rồi sinh câu trả lời — bộc lộ hạn chế trong bài toán của chúng tôi: một truy vấn diễn đạt kém sẽ trượt ngã cảnh; hệ thống không tự đánh giá mức độ liên quan để sửa sai; và nhiều thông tin "sống" mà người dân thực sự cần (địa chỉ cụ thể của cơ quan tiếp nhận, giờ làm việc, đường dây nóng theo địa bàn) vốn không nằm trong văn bản mô tả thủ tục. Cách tiếp cận **Agentic RAG** — mà tiêu biểu là mô thức ReAct [18], kết hợp suy luận và hành động — cho phép xem LLM như một tác tử biết tự quyết định: chủ động truy hỏi corpus, tự đánh giá thông tin đã đủ hay chưa, và bổ sung tìm kiếm web khi cần theo tinh thần tự sửa của Corrective-RAG [17, 1]. Cuối cùng, để người dùng tin tưởng và kiểm chứng được, trợ lý phải *trích dẫn nguồn ngay trong câu trả lời, tô sáng* đúng đoạn bằng chứng, và *minh bạch* hoá chuỗi suy luận của agent.

1.2 Phát biểu bài toán

Mục tiêu của đề án là xây dựng một trợ lý hỏi–đáp tiếng Việt cho thủ tục hành chính công, thoả các yêu cầu: (i) *có căn cứ* — mọi khẳng định phải dựa trên kho 5.208 thủ tục thu thập từ Cổng Dịch vụ công Quốc gia, hoặc nguồn web được đánh dấu rõ là chưa thẩm định; (ii) *agentic* — tự tra cứu corpus trước, bổ sung web khi corpus thiếu, hỗ trợ hội thoại đa lượt; (iii) *minh bạch* — trích dẫn nội dòng kèm tô sáng bằng chứng và hiển thị quá trình suy luận; và (iv) *linh hoạt triển khai* — chạy được với nhiều nhà cung cấp LLM khác nhau (Azure, OpenAI, mô hình mã nguồn mở phục vụ qua vLLM).

1.3 Đóng góp

Báo cáo có các đóng góp chính sau:

- **Pipeline dữ liệu đầu–cuối.** Một crawler dịch ngược (reverse-engineer) API ẩn của Cổng Dịch vụ công Quốc gia để thu thập đầy đủ 5.208 thủ tục, kèm parser chuẩn hoá JSON có cấu trúc thành corpus Markdown (frontmatter metadata + sáu mục: trình tự, cách thức, thành phần hồ sơ, điều kiện, căn cứ pháp lý, kết quả) và bước nhúng– nạp (ingest) vào kho vector/tài liệu lai ghép.
- **Kiến trúc Agentic RAG hai pha cho miền.** Pha một là vòng lặp ReAct với hai công cụ — `docsearch` (tra corpus, ưu tiên trước, có "cổng" lọc độ liên quan) và `web_search` (dự phòng qua Brave Search) — thiết kế theo hướng Corrective-RAG; pha hai tổng hợp câu trả lời có trích dẫn nội dòng.
- **Lớp trải nghiệm minh bạch.** Trực quan hoá chuỗi *Thought/Action* của agent theo thời gian thực; trích dẫn nội dòng [n] kèm tô sáng đúng đoạn bằng chứng trong nguồn; phân biệt nguồn web (ký hiệu (**web**)) chưa thẩm định; sinh sơ đồ tư duy cho thủ tục; gợi ý câu hỏi tiếp theo; và viết lại truy vấn theo ngữ cảnh hội thoại.
- **Hệ thống tích hợp, tái lập được.** Backend FastAPI phát trực tiếp (SSE) ghép với giao diện React, xây trên nền kotaemon [2] đã được vá lỗi và đóng gói kèm repo; tài liệu hoá để chạy lại từ đầu.

Phần còn lại của báo cáo được tổ chức như sau. Mục 2 điểm lại các công trình liên quan. Mục 3 trình bày phương pháp (sẽ hoàn thiện). Mục 4 mô tả kế hoạch triển khai bốn tuần kèm tiến độ.

2 Related Work

Nền tảng RAG và hỏi–đáp miền mở. RAG [7] kết hợp một bộ truy hồi với một bộ sinh để trả lời các tác vụ đòi hỏi nhiều tri thức, và đã được tổng hợp lại trong khảo sát của Gao et al. [3] theo ba thể hệ (naive, advanced, modular). Các kiến trúc liên quan như Fusion-in-Decoder [4] cho thấy lợi ích của việc đưa nhiều đoạn truy hồi vào bộ sinh. Hệ thống của chúng tôi kế thừa khung này nhưng đặt trong một vòng lặp tác tử thay vì truy hồi một lần.

Truy hồi: dày đặc, thưa, và xấp xỉ láng giềng. Dense Passage Retrieval [6] và Sentence-BERT [11] mở đường cho truy hồi theo ngữ nghĩa bằng vector nhúng, trong khi BM25 [12] vẫn mạnh ở so khớp từ khoá — đặc biệt quan trọng với các thực thể "cứng" như mã mẫu đơn (CC01) hay số văn bản (Thông tư 17/2024/TT-BCA). Vì vậy chúng tôi dùng *truy hồi lai ghép* (vector + toàn văn). Ở quy mô hàng nghìn tài liệu, tìm kiếm láng giềng gần đúng bằng đồ thị HNSW [9] cho độ trễ thấp mà vẫn giữ chất lượng.

Suy luận và tác tử dùng công cụ. Chain-of-Thought [15] cho thấy việc "suy luận thành lời" cải thiện khả năng giải quyết vấn đề của LLM. ReAct [18] đan xen *suy luận* với *hành động* (gọi công cụ) và quan sát kết quả, tạo nên khung tác tử mà chúng tôi áp dụng. Toolformer [13] chứng minh LLM có thể tự học cách dùng công cụ, còn ReWOO [16] tách lập kế hoạch khỏi quan sát để giảm số lần gọi LLM — một phương án thay thế mà kotaemon cũng hỗ trợ và chúng tôi có thể đối chiếu.

RAG thích nghi và tự sửa. Self-RAG [1] huấn luyện mô hình tự quyết định khi nào cần truy hồi và tự phê bình câu trả lời, trong khi Corrective-RAG [17] đánh giá chất lượng tài liệu truy hồi và kích hoạt tìm kiếm web khi tài liệu nội bộ không đủ tốt. Thiết kế "corpus trước, web dự phòng" cùng cơ chế cổng lọc độ liên quan của chúng tôi lấy cảm hứng trực tiếp từ hai hướng này, nhưng hiện thực bằng *prompt* trên tác tử ReAct thay vì huấn luyện lại mô hình.

Tính trung thực, ảo giác và trích dẫn. Ảo giác là rào cản cốt lõi khi đưa LLM vào miền pháp lý–hành chính [5]; ngoài ra LLM còn "bỏ quên thông tin ở giữa" khi ngữ cảnh dài [8]. Đây là lý do chúng tôi nhấn mạnh trích dẫn nội dòng kèm tô sáng đúng đoạn bằng chứng, đồng thời ràng buộc bằng *prompt* để hệ thống chỉ trả lời dựa trên ngữ cảnh đã truy hồi và nói rõ khi thông tin thiếu.

Khung công cụ và bối cảnh tiếng Việt. Đã có những khung RAG mã nguồn mở như kotaemon [2] giúp rút ngắn thời gian dựng hệ thống, song phần lớn được thiết kế và đánh giá cho tiếng Anh. So với hỏi–đáp pháp lý/hành chính tiếng Anh, tài nguyên và hệ thống cho thủ tục hành chính *tiếng Việt* còn rất ít. Đóng góp của chúng tôi nằm ở toàn bộ chuỗi xử lý cho miền này — từ thu thập và chuẩn hoá dữ liệu, thiết kế *prompt* theo đặc thù pháp lý hành chính, tới chiến lược suy luận agentic có trích dẫn — được trình bày ở Mục 3.

3 Method

Hệ thống được tổ chức thành ba thành phần kế tiếp nhau. *Thứ nhất*, một pipeline tri thức biến dữ liệu thủ tục thô thành một kho biểu diễn truy hồi được (Mục 3.1). *Thứ hai*, một lõi truy hồi–sinh có trích dẫn biến cặp (câu hỏi, kho tri thức) thành câu trả lời có dẫn nguồn (Mục 3.2). *Thứ ba*, một bộ máy suy luận đa chiến lược cho phép thay đổi cách hệ thống lập kế hoạch và sử dụng công cụ mà không phải viết lại lõi nói trên (Mục 3.3).

3.1 Thu thập dữ liệu và biểu diễn tri thức

Nguồn dữ liệu và giao diện truy cập. Toàn bộ tri thức của hệ thống được rút trích từ Cổng Dịch vụ công Quốc gia (dichvucong.gov.vn) [14] — đầu mối chính thức công bố thủ tục hành chính trên phạm vi toàn quốc. Cổng được triển khai dưới dạng ứng dụng trang đơn (single-page application), trong đó giao diện và dữ liệu tách rời nhau: nội dung thủ tục không hiện diện trong mã HTML tĩnh mà được nạp động lúc chạy thông qua một lớp dịch vụ REST nội bộ không được tài liệu hoá công khai. Hệ quả là các kỹ thuật rút trích dựa trên phân tích cây HTML (HTML scraping) gần như không khả dụng. Thay vào đó, chúng tôi tiến hành phân tích lưu lượng mạng của ứng dụng nhằm khôi phục đặc tả của các điểm cuối cần thiết. Quá trình này xác định một giao diện ổn định gồm hai điểm cuối ở chế độ công khai (hậu tố `by-citizen`, thuộc cơ sở `/api/v1`), không yêu cầu xác thực và trả về dữ liệu JSON theo một lớp vỏ thống nhất (mã trạng thái kèm trường dữ liệu); Bảng 1 tóm tắt hai điểm cuối này.

Điểm cuối (POST)	Tham số chính	Vai trò
<code>.../list-all-formality-by-citizen</code>	<code>lastId</code> , <code>limit</code>	Liệt kê danh mục thủ tục theo phân trang con trỏ
<code>.../get-formality-by-citizen</code>	<code>id</code> (UUID)	Tải bản ghi chi tiết đầy đủ của một thủ tục

Table 1: Hai điểm cuối REST công khai dùng cho thu thập dữ liệu, khôi phục từ việc phân tích lưu lượng của Cổng Dịch vụ công Quốc gia.

Quy trình thu thập hai giai đoạn. Việc thu thập được tổ chức thành hai giai đoạn nối tiếp. *Giai đoạn liệt kê* duyệt toàn bộ danh mục thủ tục qua điểm cuối danh sách theo cơ chế *phân trang con trỏ* (cursor-based pagination): thay vì truy cập theo chỉ số trang, mỗi yêu cầu mang theo định danh của phần tử cuối thuộc trang trước (`lastId`) làm mốc để máy chủ trả về lô kế tiếp, và vòng lặp dừng khi nhận được danh sách rỗng. So với phân trang theo chỉ số, cơ chế này hạn chế hiện tượng trùng lặp hoặc bỏ sót khi danh mục bị thay đổi ngay trong lúc duyệt. Đầu ra của giai đoạn là một chỉ mục gọn (định danh, mã, tên, lĩnh vực, cơ quan) cho từng thủ tục. *Giai đoạn chi tiết* lần lượt gọi điểm cuối chi tiết theo mỗi định danh để tải về bản ghi đầy đủ, lưu thành một tệp JSON riêng cho từng thủ tục.

Tính bền vững và tôn trọng máy chủ. Vì quy mô lên tới hàng nghìn yêu cầu, bộ thu thập được thiết kế để *chịu lỗi* và *tiết chế tải*. Giữa các yêu cầu, hệ thống chèn một khoảng trễ cơ sở cộng thêm nhiễu ngẫu nhiên (jitter) nhằm tránh tạo tải đột biến lên máy chủ; khi gặp lỗi mạng hoặc lỗi phía máy chủ, yêu cầu được thử lại với thời gian chờ tăng theo hàm mũ (exponential backoff). Mỗi bản ghi được ghi xuống đĩa theo cơ chế *nguyên tử* — xuất ra tệp tạm rồi đổi tên — để bảo đảm không phát sinh tệp JSON hỏng khi tiến trình bị gián đoạn. Trạng thái phân trang cùng tập định danh đã tải được lưu lại, cho phép *tiếp tục* (resume) sau khi dừng mà không xử lý lại phần đã hoàn tất; các định danh thất bại được ghi nhật ký riêng để thử lại về sau. Dữ liệu thu thập là thông tin công khai, được sử dụng cho mục đích học tập và nghiên cứu.

Kết quả và đặc tả bản ghi. Quy trình trên thu về **5.208** bản ghi thủ tục (tính đến tháng 06/2026). Mỗi bản ghi là một tài liệu JSON có cấu trúc, mô tả đầy đủ một thủ tục trên các trường: trình tự thực hiện, cách thức thực hiện (kèm thời hạn giải quyết và mức phí, lệ phí), thành phần hồ sơ, yêu cầu–điều kiện, căn cứ pháp lý, kết quả, lĩnh vực, cơ quan thực hiện và đối tượng áp dụng. Đáng chú ý, thành phần hồ sơ — trường thông tin quan trọng nhất đối với người dùng cuối — không nằm ở cấp cao nhất của bản ghi mà lồng bên trong các *trường*

hợp thực hiện (*executionCases*); việc xác định đúng vị trí lồng nhau này là điều kiện tiên quyết để bước chuẩn hoá liệt kê đầy đủ giấy tờ, tránh bỏ sót. Cấu trúc giàu metadata của bản ghi đồng thời là cơ sở cho việc lọc theo lĩnh vực hoặc cấp thực hiện và cho việc truy vết nguồn ở các bước sau.

Chuẩn hoá và phân đoạn. Mỗi bản ghi JSON được chuyển thành một tài liệu Markdown tự chứa gồm hai phần: một khối *metadata* (mã thủ tục, lĩnh vực, cấp thực hiện, cơ quan, đối tượng) phục vụ lọc và truy vết, và sáu *mục nội dung* đã làm sạch. Thay vì cắt văn bản theo độ dài cố định, chúng tôi phân đoạn *theo mục* (section-based chunking): mỗi mục trở thành một đơn vị truy hồi có ngữ nghĩa trọn vẹn, được gắn tiền tố **[Tên thủ tục -- Mục]** ở đầu. Cách phân đoạn này giúp tránh tình trạng dữ kiện quan trọng (thành phần hồ sơ, lệ phí) bị xé lẻ giữa nhiều đoạn, đồng thời cho phép định danh nguồn ngay từ nội dung chunk.

Nhúng và lưu trữ. Mỗi đơn vị truy hồi được nhúng thành một vector **3072 chiều** bằng mô hình **text-embedding-3-large**. Để rút ngắn thời gian xử lý hàng nghìn đoạn, bước nhúng được thực thi *song song* nhiều luồng, gộp lô và thử lại theo hàm mũ khi gặp giới hạn tần suất của dịch vụ. Kho tri thức gồm ba lớp được giữ *đồng bộ* (Hình 1): (a) một chỉ mục vector dạng đồ thị HNSW [9] cho tìm kiếm theo ngữ nghĩa; (b) một chỉ mục từ khoá toàn văn cho so khớp các thực thể “cứng” như mã mẫu đơn (CC01) hay số văn bản (Thông tư 17/2024/TT-BCA), nơi truy hồi thuần ngữ nghĩa thường kém; và (c) một catalog quan hệ ánh xạ tài liệu ↔ đoạn ↔ vector để truy vết nguồn về sau.

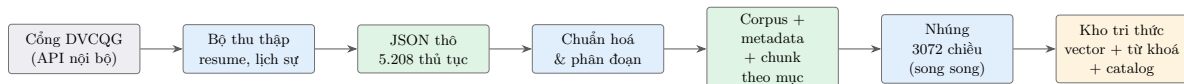


Figure 1: Pipeline tri thức: từ API của Cổng Dịch vụ công Quốc gia tới kho truy hồi ba lớp.

3.2 Truy hồi và sinh câu trả lời có trích dẫn

Thành phần này nhận một câu hỏi (kèm lịch sử hội thoại) và trả về câu trả lời có trích dẫn nội dòng. Nó vận hành theo bốn bước (Hình 2).

(1) Viết lại truy vấn theo ngữ cảnh. Câu hỏi ngắn thường là câu nối tiếp thiếu ngữ cảnh (“còn quy trình thì sao?”) hoặc lời chào. Hệ thống dùng lịch sử hội thoại để viết lại thành một truy vấn *độc lập, tự đủ nghĩa*, đồng thời giải nghĩa các từ viết tắt phổ biến (cccd, mst, bxxh...). Nếu phát hiện tin nhắn chỉ là chào hỏi xã giao, bước này phát tín hiệu bỏ qua truy hồi để hệ thống đáp lại tự nhiên thay vì tra cứu.

(2) Truy hồi lai ghép. Truy vấn được đưa song song qua hai kênh: tìm kiếm *ngữ nghĩa* trên chỉ mục vector và tìm kiếm *từ khoá* trên chỉ mục toàn văn; kết quả hai kênh được hợp nhất. Sự kết hợp này tận dụng điểm mạnh bù trừ của hai họ phương pháp [6, 12]: ngữ nghĩa cho câu hỏi diễn đạt tự do, từ khoá cho mã hiệu và số văn bản chính xác.

(3) Cổng lọc độ liên quan. Khác với chiến lược luôn lấy k đoạn gần nhất — vốn *luôn* trả về kết quả ngay cả khi các đoạn không thực sự liên quan — mỗi đoạn truy hồi được một LLM đánh giá mức độ liên quan với câu hỏi, và những đoạn dưới ngưỡng sẽ bị loại bỏ. Cơ chế “cổng” này tạo ra một tín hiệu *thiếu thông tin* tường minh khi không còn đoạn nào sống sót; tín hiệu đó chính là nền tảng cho hành vi tự sửa của chiến lược agentic ở Mục 3.3.

(4) **Tổng hợp có trích dẫn và tô sáng.** Các đoạn còn lại được ghép thành ngữ cảnh và đưa vào một LLM với prompt buộc trả lời theo *hai khối*: một khối liệt kê trích dẫn — mỗi nguồn một số thứ tự [n] kèm cặp **START_PHRASE/END_PHRASE** sao y nguyên văn đoạn bằng chứng — và một khối câu trả lời, trong đó số [n] được chèn ngay sau mỗi dữ kiện. Hệ thống dò lại cặp cụm từ trong nguồn để **tô sáng** đúng đoạn bằng chứng và biến mỗi [n] thành liên kết tới nguồn. Toàn bộ prompt được ràng buộc theo đặc thù miền: chỉ dùng thông tin trong ngữ cảnh, giữ nguyên văn tên giấy tờ và mã mẫu, nêu rõ phần thông tin còn thiếu thay vì suy diễn — nhằm kìm hãm hiện tượng ảo giác [5]. Khi điểm liên quan tổng thể thấp, hệ thống hiển thị cảnh báo để người dùng thận trọng kiểm chứng. Tùy chọn, một sơ đồ tư duy (mind map) được sinh kèm để tóm tắt thủ tục dưới dạng phân cấp.

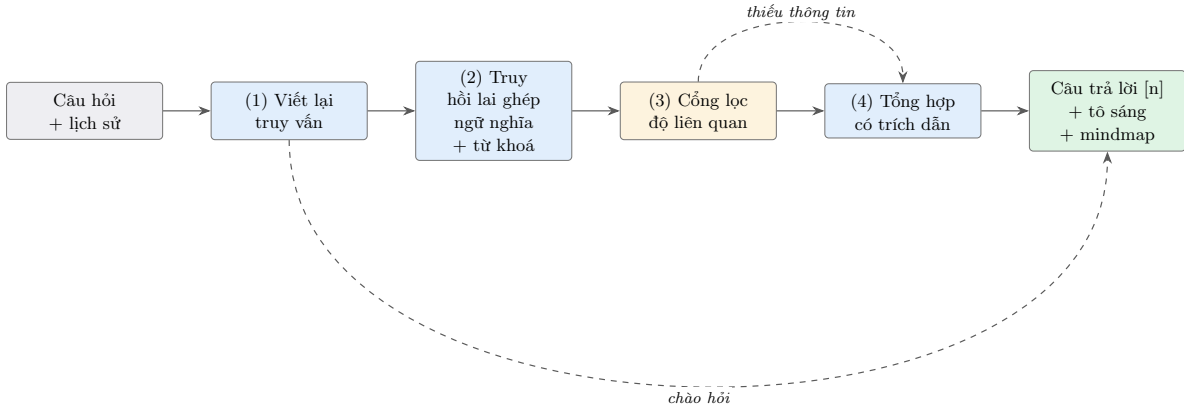


Figure 2: Lỗi truy hồi–sinh có trích dẫn (suy luận một lượt). Nhánh nét đứt: bỏ qua truy hồi khi chỉ là chào hỏi, và tín hiệu thiếu thông tin khi cổng lọc loại hết các đoạn.

3.3 Tích hợp các chiến lược suy luận

Suy luận một lượt ở Mục 3.2 đủ cho các câu hỏi trực diện, nhưng bộc lộ hạn chế khi (i) câu hỏi cần nhiều bước hoặc trải trên nhiều thủ tục, và (ii) thông tin người dùng cần không nằm trong corpus — ví dụ địa chỉ cụ thể, giờ làm việc hay đường dây nóng của cơ quan tiếp nhận tại một địa bàn. Để mở rộng mà không phá vỡ lõi đã có, chúng tôi định nghĩa một *giao diện suy luận chung*: một chiến lược nhận đầu vào là (câu hỏi, lịch sử, tập bộ truy hồi) và phát ra một luồng câu trả lời. Mọi chiến lược cài đặt cùng giao diện này và *dùng chung* bước tổng hợp có trích dẫn ở Mục 3.2 cho khâu sinh câu trả lời; việc chọn chiến lược chỉ là một mục cấu hình lúc chạy. Chúng tôi tích hợp ba chiến lược.

Chiến lược A — Suy luận một lượt (Simple). Chính là pipeline ở Mục 3.2: một lần truy hồi, một lần sinh. Đây là đường cơ sở (baseline) có chi phí tính toán và độ trễ thấp, phù hợp với các câu hỏi trực diện về một thủ tục đơn lẻ.

Chiến lược B — Tác tử ReAct hai pha (mặc định). Chiến lược này hiện thực mô thức ReAct [18] và chạy theo hai pha (Hình 3). Với hội thoại đa lượt, câu hỏi được *contextualize* (giải tham chiếu ngầm như “thủ tục này”, “cơ quan ấy”) trước khi vào pha một.

Pha 1 — gom nguồn. Tác tử lặp chu trình *Thought* → *Action* → *Observation*. Nó có hai công cụ: **docsearch** (tra corpus, có cổng lọc liên quan như Mục 3.2) và **web_search** (tìm trên internet). Một *quy tắc corpus trước* được áp đặt qua prompt: với mọi câu hỏi, hành động đầu tiên phải là **docsearch**; chỉ khi corpus báo “không có tài liệu liên quan”, hoặc khi corpus chỉ trả lời chung chung mang tính dẫn chiếu mà câu hỏi lại cần chi tiết cụ thể theo địa bàn, tác

tử mới gọi `web_search` để *bổ sung*. Đây chính là tinh thần Corrective-RAG [17, 1] được hiện thực bằng prompt cộng với tín hiệu từ cổng lọc liên quan, thay vì huấn luyện lại mô hình. Mỗi đoạn corpus liên quan và mỗi kết quả web (được bọc thành nguồn trích dẫn được, gắn nhãn web) được dồn vào một “bể gom nguồn” dùng chung.

Pha 2 — tổng hợp. Tập nguồn đã gom được đưa qua *đúng* lõi tổng hợp có trích dẫn ở Mục 3.2, sinh ra câu trả lời với các trích dẫn [n]; những trích dẫn trở về nguồn web được tự động đánh dấu kèm liên kết để phân biệt với nguồn chính thống. Trong suốt pha một, từng bước *Thought/Action/Observation* được phát trực tiếp ra giao diện, giúp người dùng thấy được agent đã suy luận và tra cứu những gì — một yêu cầu minh bạch quan trọng trong miền hành chính.

Chiến lược C — Lập kế hoạch tách rời (*ReWOO*). Như một phương án đánh đổi khác, chúng tôi tích hợp ReWOO [16]: thay vì đan xen suy luận với quan sát ở mỗi bước, tác tử lập *toàn bộ* kế hoạch gọi công cụ trước, rồi mới thực thi và tổng hợp. Cách này giảm số lần gọi LLM (vì không phải nạp lại toàn bộ lịch sử quan sát ở mỗi vòng), phù hợp khi ưu tiên chi phí và độ trễ. Bảng 2 đối chiếu ba chiến lược.

Chiến lược	Số lượt truy hồi	Web dự phòng	Tự sửa	Phù hợp
Simple (một lượt)	1	Không	Không	Câu hỏi trực diện, một thủ tục
ReAct (mặc định)	Lặp ($\leq N$)	Có	Có	Câu hỏi phức hợp; cần thông tin ngoài corpus
ReWOO	Theo kế hoạch	Có	Hạn chế	Tối ưu chi phí khi dùng nhiều công cụ

Table 2: So sánh ba chiến lược suy luận. Cả ba dùng chung lõi truy hồi và lõi tổng hợp có trích dẫn (Mục 3.2); chỉ khác ở cách lập kế hoạch và gọi công cụ.

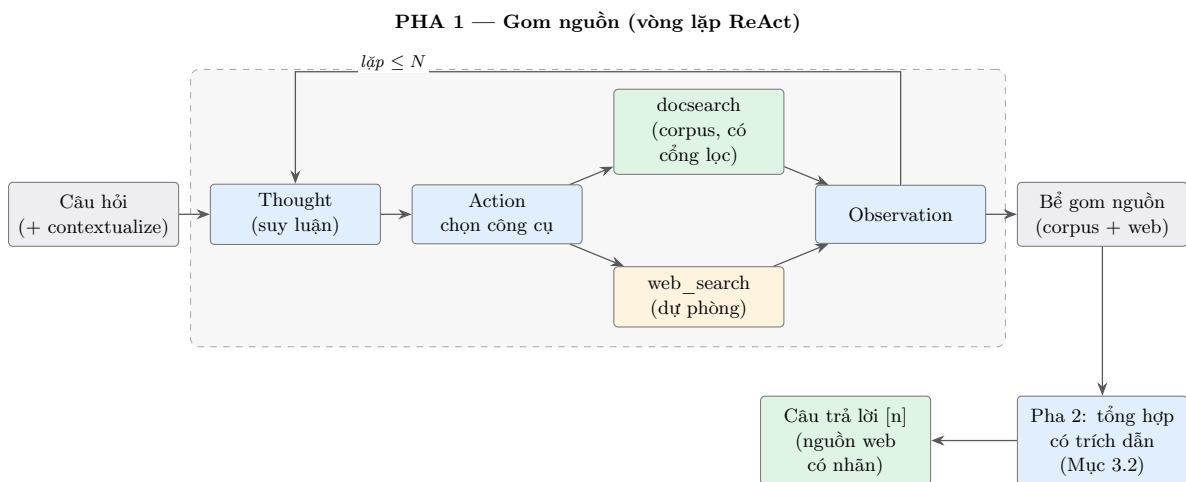


Figure 3: Chiến lược agentic ReAct hai pha. Pha 1 lặp *Thought/Action/Observation* để gom nguồn từ corpus (ưu tiên) và web (dự phòng, theo tinh thần Corrective-RAG); pha 2 tái sử dụng lõi tổng hợp có trích dẫn ở Mục 3.2.

Nguyên tắc thiết kế *một lõi truy hồi–sinh dùng chung, nhiều chiến lược suy luận khả cảm* mang lại hai lợi ích. Thứ nhất, chất lượng trích dẫn và độ trung thực của câu trả lời được bảo đảm *đồng nhất* trên mọi chế độ suy luận. Thứ hai, việc bổ sung một chiến lược mới về sau chỉ đòi hỏi hiện thực giao diện chung, không tác động tới tầng dữ liệu hay khâu sinh câu

trả lời có trích dẫn.

4 Kế hoạch thực hiện

Đồ án được triển khai trong khoảng **29/5–30/6/2026**, chia thành bốn tuần với mục tiêu tăng dần từ dữ liệu → baseline → agentic → tổng quát hoá & báo cáo. Bảng 3 tóm tắt lộ trình. Trong các danh sách bên dưới, ô ☒ là hạng mục đã hoàn thành tính đến thời điểm viết báo cáo (12/6/2026), ô ☐ là hạng mục đang hoặc sẽ thực hiện.

Hạng mục công việc	Tuần 1	Tuần 2	Tuần 3	Tuần 4
	29/5–5/6	6–12/6	13–22/6	23–30/6
GĐ 1 — Dữ liệu & pipeline nhúng Crawl, chuẩn hoá, nhúng, dựng vector DB	☒	☐		
GĐ 2 — Khảo sát RAG & baseline Survey RAG, prompt domain, chế độ <i>simple</i>		☒		
GĐ 3 — Agentic RAG & tích hợp ReAct 2 pha, API+UI, citation/highlight, gợi ý		☒	☒	☒
GĐ 4 — Đa LLM & báo cáo Azure/OpenAI/vLLM, viết báo cáo PDF				☒

Table 3: Lộ trình thực hiện theo tuần (29/5 – 30/6/2026). Ô đậm = tuần trọng tâm của giai đoạn; ô nhạt = phần việc gởi đầu sang tuần lân cận.

4.1 Tuần 1 (29/5 – 5/6): Dữ liệu & pipeline nhúng đầu–cuối

- ☒ Khảo sát nguồn dữ liệu và dịch ngược API ẩn của Cổng Dịch vụ công Quốc gia (endpoint danh sách + chi tiết theo công dân).
- ☒ Viết crawler có khả năng resume, lịch sự (delay + jitter, retry kèm backoff), ghi file nguyên tử; thu thập **5.208 thủ tục** (`data/raw/*.json`).
- ☒ Viết parser JSON → Markdown: frontmatter metadata (mã, lĩnh vực, cấp thực hiện, cơ quan, đối tượng) + sáu mục nội dung; xuất kèm `chunks.jsonl` theo từng mục.
- ☒ Cấu hình nhúng Azure `text-embedding-3-large` (3072 chiều), ngôn ngữ tiếng Việt.
- ☒ Nạp corpus vào kho vector + kho tài liệu lai ghép (Chroma + LanceDB); bản nạp song song nhiều worker để tăng tốc.
- ☒ Đóng gói và chia sẻ index (HuggingFace) để máy khác tái lập không cần nhúng lại.

4.2 Tuần 2 (6/6 – 12/6): Khảo sát RAG & baseline *simple*

- ☒ Khảo sát các phương pháp RAG (naive/advanced/modular; truy hồi–rerank; viết lại truy vấn; tìm kiếm lai ghép) và chọn thiết kế phù hợp miền.

- ☒ Thiết kế prompt QA tiếng Việt cho miền (giữ nguyên văn tên giấy tờ/mã mẫu/số văn bản; liệt kê hồ sơ kèm số bản chính/bản sao).
- ☒ Viết lại truy vấn: giải nghĩa viết tắt (cccd, mst, bxxh...), bỏ qua lời chào xã giao.
- ☒ Triển khai baseline *simple* trên UI: viết lại truy vấn → truy hồi lai ghép → rerank → sinh câu trả lời kèm nguồn.
- ☐ Đánh giá định tính baseline trên một tập câu hỏi mẫu để làm mốc so sánh.

4.3 Tuần 3 (13/6 – 22/6): Agentic RAG, tích hợp & minh bạch hoá

- ☒ Thiết kế kiến trúc Agentic RAG hai pha dựa trên ReAct.
- ☒ Pha 1 — vòng lặp ReAct với công cụ `docsearch` (corpus, ưu tiên trước, có cổng lọc độ liên quan) và `web_search` (Brave, dự phòng, có tải nội dung trang).
- ☒ Prompt ReAct tiếng Việt giữ nguyên từ khoá *Thought/Action/Observation*; quy tắc corpus-first và bổ sung web khi corpus chỉ trả lời chung chung/dẫn chiếu.
- ☒ Pha 2 — tổng hợp câu trả lời có trích dẫn nội dòng [n] kèm cặp START/END_PHRASE để **tô sáng** đúng đoạn bằng chứng.
- ☒ Phân biệt nguồn web (**web**) chưa thẩm định trong panel nguồn và nhắc người dùng kiểm chứng.
- ☒ Trực quan hoá quá trình suy luận: phát các bước *Thought/Action* theo thời gian thực tới UI qua SSE.
- ☒ Gợi ý câu hỏi tiếp theo (follow-up) dựa trên lịch sử hội thoại.
- ☒ Hỗ trợ hội thoại đa lượt: viết lại câu hỏi theo ngữ cảnh (giải tham chiếu "thủ tục này", "nó"...).
- ☒ Sinh sơ đồ tư duy (PlantUML) cho thủ tục, gom nhóm giấy tờ bắt buộc/theo trường hợp.
- ☒ Tích hợp backend FastAPI (streaming SSE) + frontend React (khung chat, panel nguồn, bảng suy luận, ô gợi ý, nút regen/stop).
- ☐ Đánh giá hệ thống agentic so với baseline (độ chính xác, độ phủ nguồn, tính trung thực) và phân tích lỗi.

4.4 Tuần 4 (23/6 – 30/6): Tổng quát hoá đa LLM & báo cáo

- ☐ Trừu tượng hoá tầng LLM/embedding theo cấu hình, không phụ thuộc một nhà cung cấp.
- ☐ Bổ sung adapter cho Azure OpenAI (hiện có), OpenAI, và mô hình mã nguồn mở phục vụ qua vLLM (endpoint tương thích OpenAI).
- ☐ Cho phép chọn mô hình qua `.env`/trang Settings mà không phải sửa mã.
- ☐ Kiểm thử đầu-cuối với từng nhà cung cấp.
- ☐ Viết báo cáo PDF chi tiết hướng dẫn cách làm (kiến trúc, pipeline, cấu hình, cách tái lập) và hoàn thiện phần *Method* của báo cáo này.

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Cinnamon AI. kotaemon: An open-source RAG-based tool for chatting with your documents. <https://github.com/Cinnamon/kotaemon>, 2024. Truy cập 06/2026.
- [3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [4] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 874–880, 2021.
- [5] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [8] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:157–173, 2024.
- [9] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [10] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [11] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [12] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] Văn phòng Chính phủ Việt Nam. Cổng dịch vụ công quốc gia. <https://dichvucong.gov.vn>, 2024. Truy cập 06/2026.

- [15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
- [16] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. ReWOO: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- [17] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- [18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.